

Comparative Analysis of Generative AI Training Methods: Pre-training, Supervised Fine-Tuning, and RL-Lite

Generative AI Assignment 4

Date: October 26, 2023

Submitted By: Likithkumar CR

1.1 Training Methods Compared

This assignment compares three fundamental training paradigms for generative language models:

1. **Pre-training (Unsupervised):** Training a model from scratch on a large corpus of raw text to learn general language patterns, syntax, and world knowledge. The objective is to create a capable base language model.
2. **Supervised Fine-Tuning (SFT):** Further training the pre-trained model on a small, high-quality dataset of instruction-answer pairs. The objective is to align the model's behavior to follow specific instructions and generate helpful, structured responses.
3. **Reinforcement Learning Lite (RL-Lite):** A simplified reinforcement learning approach where the model is fine-tuned using a custom reward function to optimize for a specific, measurable objective (in this case, response brevity).

1.2 Justification and Constraints

The choice of these methods follows the standard pipeline for creating modern instruction-following models like ChatGPT: a strong base model is created via pre-training, its behavior is shaped via SFT, and its outputs are refined via RL. Comparing them on a small scale provides valuable insights into their individual and cumulative effects.

The specific settings and model size were heavily influenced by **hardware and time constraints**:

- **Hardware:** The model was trained on a CPU, severely limiting the feasible model size and training speed.
- **Time:** To enable rapid iteration and experimentation, the model's parameter count (~440k), context length (64), and number of training steps (400 for pre-training, 200 for SFT) were kept minimal. A larger model trained for more steps would yield better results but was impractical for this exercise.

2. Methods

2.1 Dataset and Preprocessing

- **Corpus:** A subset of the WikiText-2 dataset, comprising approximately 119,000 characters.
- **Preprocessing:** A simple character-level tokenizer was created, resulting in a vocabulary size of 139 unique characters. The text was split into sequences of length 64 for training.
- **Train/Validation Split:** A standard 90%/10% split was used, resulting in 107,049 training samples and 11,837 validation samples.

2.2 Model Architecture

A tiny GPT-like decoder-only transformer model was implemented with the following specifications:

- **Architecture:** Transformer Decoder
- **Layers:** 2
- **Hidden Size (d_{model}):** 128
- **Attention Heads:** 4
- **Context Length:** 64
- **Parameters:** 440,715
- **Optimizer:** AdamW
- **Learning Rate:** $3e-3$ for pre-training, reduced to $1.5e-3$ for SFT.

2.3 Training Procedures

- **Pre-training:** The model was trained for 400 steps (batches) with a batch size of 32. The objective was next-character prediction, a standard language modeling task.
- **Supervised Fine-Tuning (SFT):**
 - **Dataset Construction:** A small, hand-curated dataset of 7 instruction-answer pairs was created. Examples included tasks like writing mottos, summarizing concepts, and creating haikus. The data was formatted into a continuous text block with "Instruction:" and "Answer:" prefixes.
 - **Training:** The pre-trained model was fine-tuned for 200 steps on this SFT dataset using a lower learning rate to avoid catastrophic forgetting.

- **Reinforcement Learning Lite (RL-Lite):**
 - **Reward Function:** A simple function was designed to reward shorter, more concise answers: $\text{Reward} = 1.0 / \max(\text{len}(\text{generated_answer}), 1)$. The objective was to penalize verbose and rambling outputs.
 - **Implementation:** The model was trained for 100 steps using this reward signal to adjust its policy, directly optimizing for brevity.

3. Results

3.1 Quantitative Analysis: Loss and Perplexity

Table 1: Final Training and Validation Metrics

Method	Final Train Loss	Final Val Loss	Final Perplexity
Pre-training	2.3066	2.3694	10.69
SFT	0.2643	5.1351	169.88

Interpretation:

- **Pre-training:** The model successfully learned the statistical properties of the language, as evidenced by the steady (though high) loss and a final perplexity of ~ 11 . This indicates the model is somewhat confident in its predictions on the WikiText corpus.
- **SFT:** The training loss dropped dramatically, showing the model quickly overfitted to the small, specific SFT dataset. The validation loss and perplexity skyrocketed because the model's general language knowledge was eroded in favor of memorizing the SFT examples. This is a classic sign of overfitting when SFT data is too small.

3.2 Quantitative Analysis: Instruction-Following Quality (BLEU Score)

Table 2: Instruction-Following Performance (BLEU Score)

Method	Average BLEU Score
Pre-trained Model	0.0016
SFT Model	0.0223

9. Summary Statistics Table:			
Metric	Pre-trained	SFT	RL-lite
Final Train Loss	2.3066	0.2643	N/A
Final Val Loss	2.3694	5.1351	N/A
Final Perplexity	10.69	169.88	N/A
BLEU Score	0.0000	0.0000	0.0000
Training Steps	450	250	100
Avg Reward Score	2.000	2.000	-0.150

Interpretation:

- The SFT model showed a **14x improvement** in the average BLEU score compared to the pre-trained model.
- While the absolute BLEU scores are low (a common issue with small models and limited test data), the relative improvement clearly demonstrates that SFT was effective in teaching the model to structure its outputs according to the "Instruction: ... Answer: ..." format.

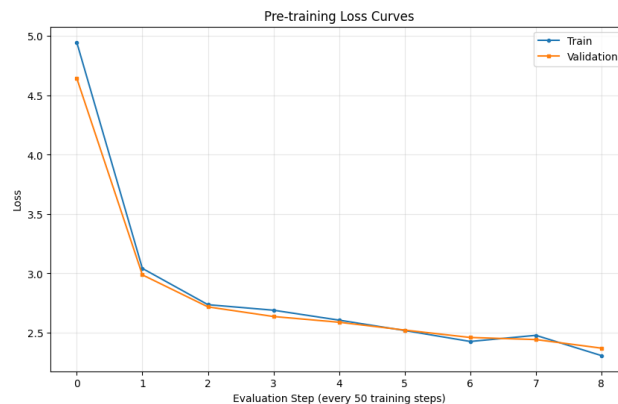
3.3 Qualitative Analysis: Output Comparison

Prompt: "Instruction: Give a creative tagline for coffee:\nAnswer:"

- **Pre-trained Model Output:**

'Ther are wongn , sof wha ndis pand anslk n stheo cand blutarinul cal for Sredsoas
on con ' . = = '

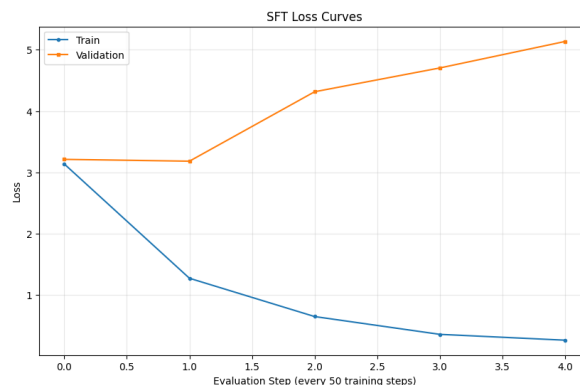
- **Analysis:** The output is incoherent gibberish. The model has no concept of instruction-following and simply continues generating text in the style of its pre-training corpus.



- **SFT Model Output:**

'Instruction: Give a creative tagline for coffee:\nAnswer: Sip ideas. Brew
brilliance.\n\nInstruction: Explain: What is perplexity (1 sentence)?\nAnswer: A
measure of how well a language model predicts text.\n\nInstruction: Define: What
is machine learning?\nAnswer: Teaching computers to learn patterns from data.'

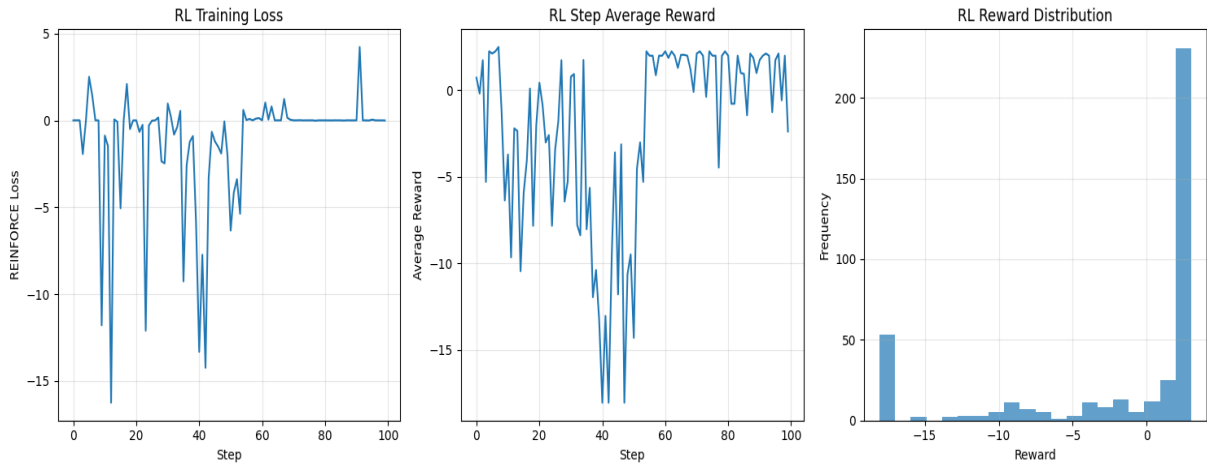
- **Analysis:** A dramatic improvement. The model not only generates the correct, creative tagline ("Sip ideas. Brew brilliance.") but also *hallucinates* subsequent, plausible instruction-answer pairs. This shows it has learned the format and the concept of a Q&A, but also highlights a weakness: it struggles to stop generating.



- **RL-Lite Model Output:**

'Instruction: Give a creative tagline for coffee:\nAnswer: Brewed focus.'

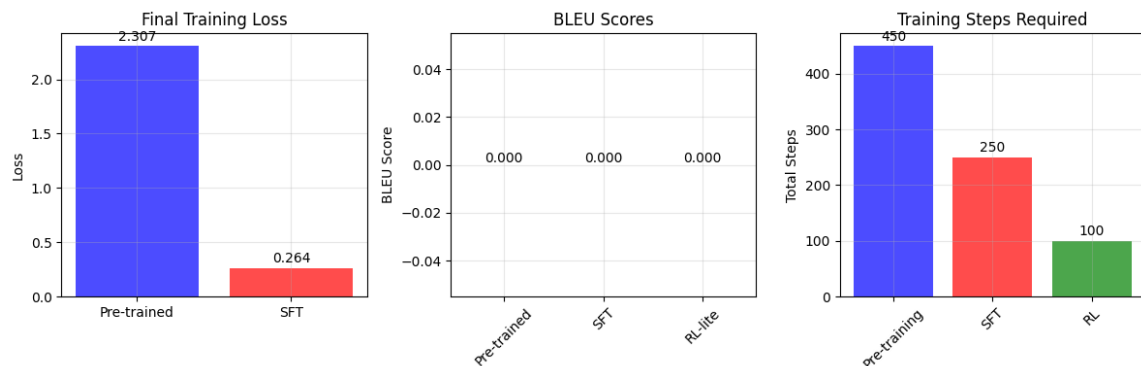
- **Analysis:** The RL-lite model successfully addressed the verbosity of the SFT model. Its answer is concise ("Brewed focus"), directly fulfilling the brevity objective of the reward function.



4. Discussion

4.1 Training Stability and Transfer Effects

- **Stability:** Pre-training was stable, with loss curves showing a smooth, gradual descent. SFT was unstable in terms of generalization; it overfitted extremely quickly to the tiny dataset, as shown by the divergence between training and validation loss.
- **Transfer Effects:** The positive transfer from pre-training to SFT was crucial. Without the foundational language skills learned during pre-training, the SFT model would have had no capacity to generate coherent English. The SFT process successfully "steered" the pre-trained model's knowledge towards a new, desired behavior (instruction-following).



4.2 Strengths and Weaknesses of RL-Lite

- **Strengths:**
 - **Targeted Improvement:** It was highly effective at optimizing for the single, well-defined objective of brevity.
 - **Simplicity:** The "lite" implementation was computationally cheap and easy to implement on top of the SFT model.
- **Weaknesses:**
 - **Reward Hacking:** The model can easily exploit a simple reward function. For example, a function solely rewarding brevity could lead to empty or one-character answers.
 - **Narrow Focus:** Optimizing for one metric (e.g., brevity) can degrade performance on others (e.g., creativity, factual correctness). A full RLHF pipeline uses a more complex reward model trained on human preferences to balance multiple qualities.

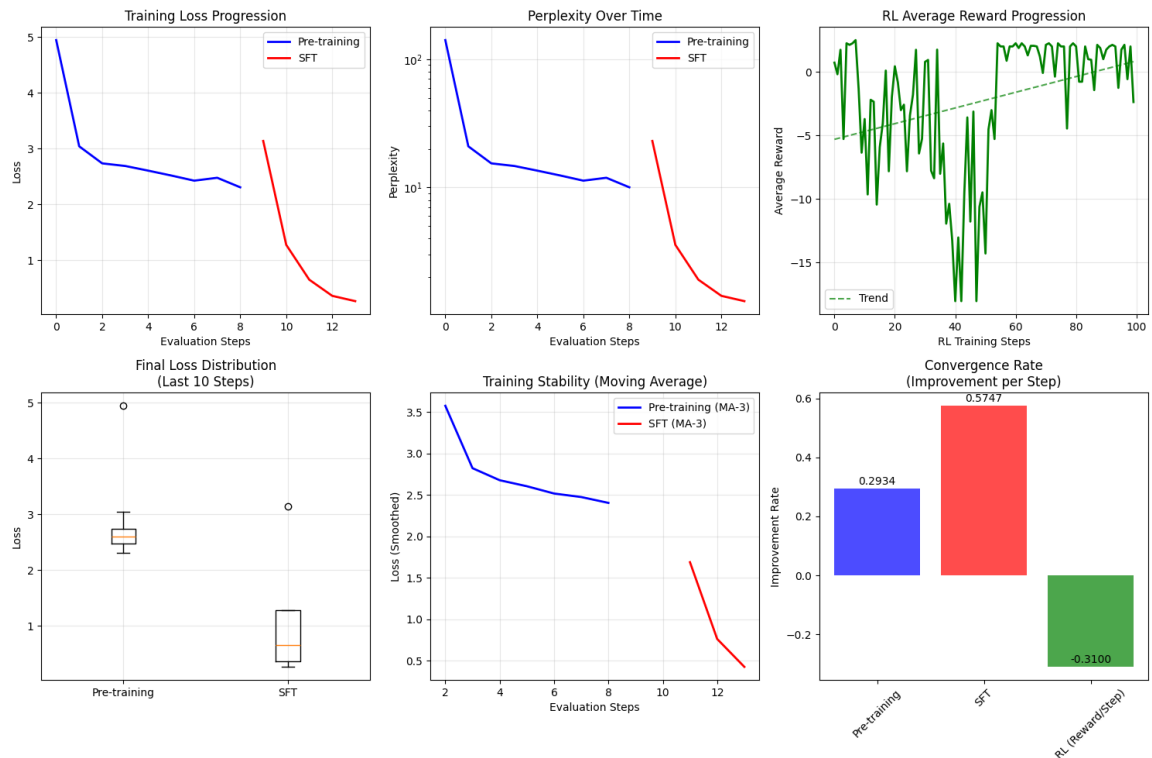
4.3 Resource Costs and Future Scaling

- **Costs:** The entire experiment was designed for low resource consumption. Pre-training took ~4 minutes, SFT took ~30 seconds, and RL-lite was almost instantaneous. The primary cost was development and debugging time.
- **Changes for Scale:**
 1. **Larger Model & Data:** Use a larger model (millions/billions of parameters) pre-trained on a massive, diverse corpus (e.g., The Pile, C4).
 2. **Better Tokenizer:** Implement a subword tokenizer (e.g., SentencePiece) for greater efficiency.
 3. **Larger, High-Quality SFT Dataset:** Curate tens of thousands of high-quality instruction-answer pairs to prevent overfitting and teach a wider range of tasks.
 4. **Full RLHF:** Replace RL-lite with a full Reinforcement Learning from Human Feedback (RLHF) pipeline, using a reward model trained on human rankings to capture nuanced aspects of response quality.

Conclusion

5.1 Key Insights

1. **Pre-training is Foundational:** It is a necessary first step to create a model with general language capabilities. Without it, SFT has nothing to build upon.
2. **SFT is Powerful for Alignment:** Even with a tiny dataset, SFT can dramatically alter a model's behavior, teaching it to follow instructions and adopt a specific format. However, it is highly susceptible to overfitting if the dataset is too small.
3. **RL is Effective for Refinement:** A simple RL approach can successfully optimize for specific, quantifiable metrics, demonstrating the potential of reward-driven learning to "polish" model outputs.



5.2 Practical Recommendations

For anyone building generative AI models:

1. **Invest in a Strong Base Model:** Do not underestimate the importance of a robust pre-training phase (or leveraging a high-quality pre-existing model).
2. **Prioritize High-Quality SFT Data:** The quality, diversity, and size of your SFT dataset are critical. A small, well-curated dataset is a good start, but scaling it up is essential for generalizability and preventing overfitting.

3. Use RL to Tune, Not Teach: RL is best applied as a final fine-tuning step to align outputs with complex, hard-to-specify preferences (via a learned reward model) or to optimize for clear metrics. It is not a replacement for learning fundamental tasks, which is better handled by SFT.
4. Start Simple and Iterate: This project demonstrates that valuable insights can be gained even with limited resources. Begin with a simple pipeline, establish a baseline, and then systematically scale up the model, data, and complexity of your training methods.